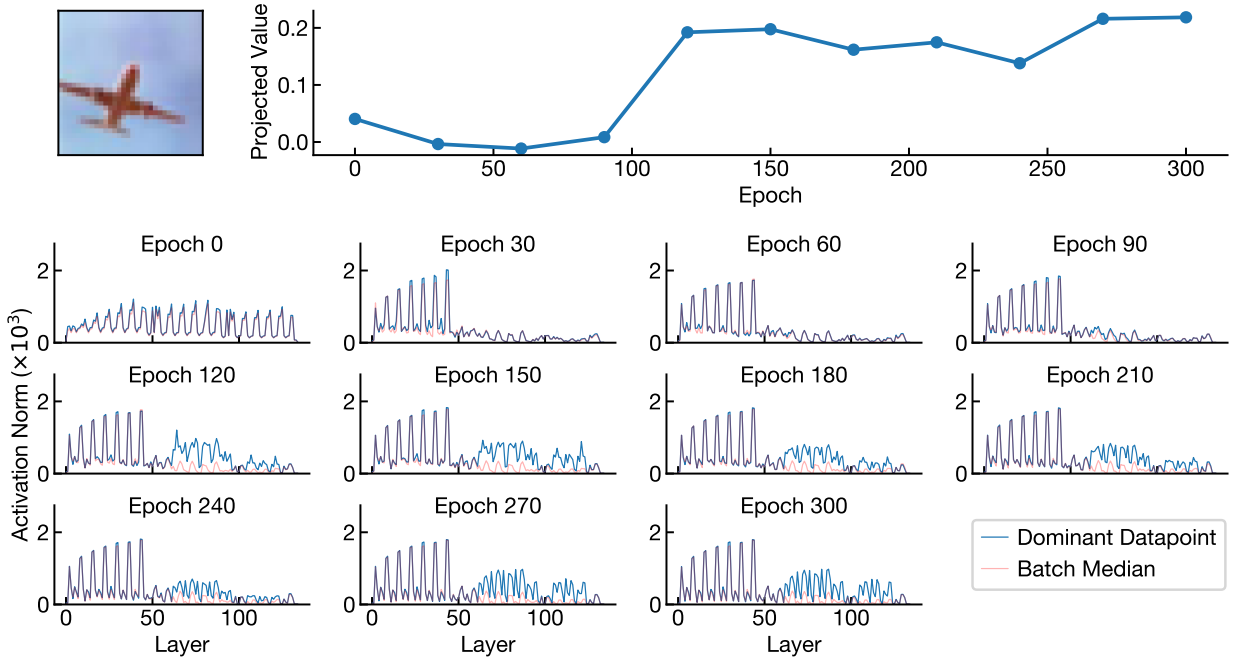

Figure F.6: Example of how the activation magnitude and the projected values onto the first principal component for a dominant image vary across different epochs, for ResNet-38 (10×). Given a dominant datapoint for a ResNet-38 (10×) model, we track the magnitude of its projected value onto the first principal component of the block structure representations (top left), as well as its activation norms at each layer in the network (bottom set of plots), over time. We observe that before the block structure representations stabilize and show more similarity with those in the fully trained model (i.e., epoch 90, see Figure F.5 above), the dominant image yields a small value when projected onto the first principal component, and also doesn't strongly activate the layers inside the block structure. This is aligned with the observation made in Figure F.4 for ResNet-164 (1×).

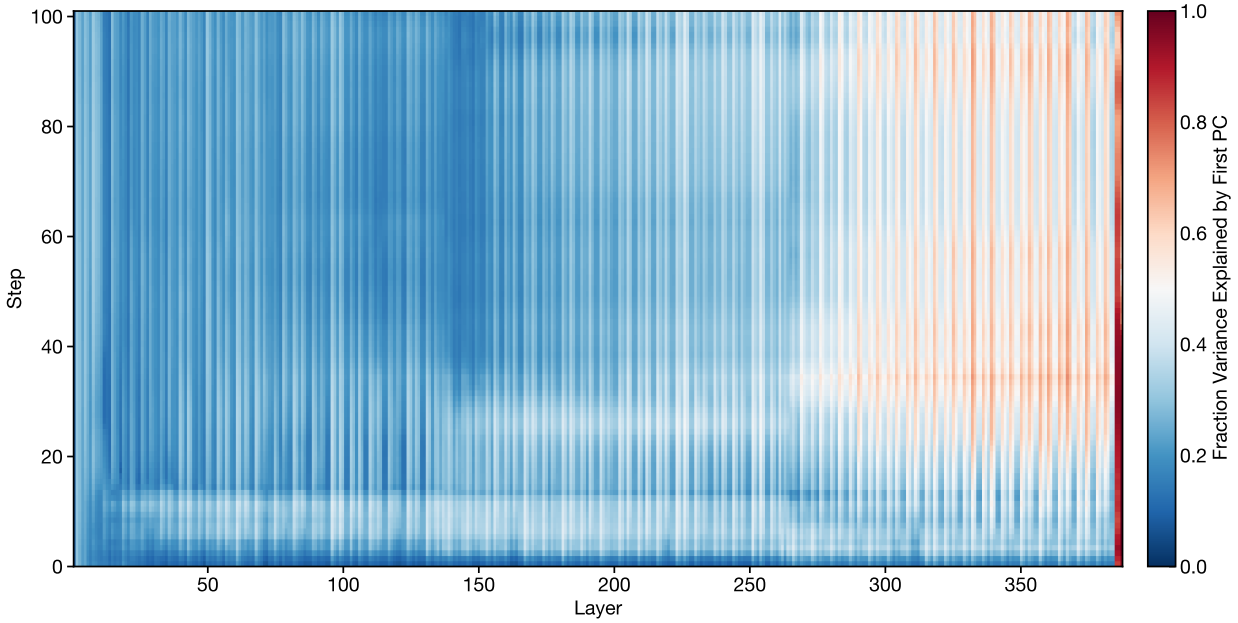

Figure F.7: Evolution of the first principal components of layer representations over the first 100 steps of training. At every step of training, we measure the proportion of variance explained by the first principal component in each layer of a ResNet-110 (1×) network. At around step 30, the first principal component begins to explain the majority of the variance in the layer representations.

G Training with Principal Component Regularization

To regularize the first principal component, we first compute the amount of variance that it explains using power iteration [25]. At training step, t, we compute the batch of n convolutional feature maps with height h and width w containing c channels $\boldsymbol{M}_t \in \mathbb{R}^{n \times h \times w \times c}$, flatten the spatial dimensions to the channels dimension to create a matrix of size $\boldsymbol{X}_t \in \mathbb{R}^{n \times p}$ where $p = h \times w \times c$, and subtract its column means to obtain a centered matrix $\tilde{\boldsymbol{X}}_t$. We randomly initialize the stored eigenvector $\boldsymbol{u}_0 \in \mathbb{R}^p$ at the beginning of training. At each training step, we perform a single step of power iteration initialized from the previous eigenvector:

$$\boldsymbol{v}_t = \tilde{\boldsymbol{X}}_t^\mathsf{T} \tilde{\boldsymbol{X}}_t \boldsymbol{u}_{t-1} \tag{3}$$

$$\lambda_t = \|\boldsymbol{v}_t\|_2 \tag{4}$$

$$\boldsymbol{u}_t = \boldsymbol{v}_t / \lambda_t. \tag{5}$$

λ_t approximates the top eigenvalue of $\tilde{\boldsymbol{X}}_t^\mathsf{T} \tilde{\boldsymbol{X}}_t$ and thus the amount of variance explained by the first principal component of the representation. The proportion of variance explained is given by $\lambda_t / \|\tilde{X}_t\|_\mathrm{F}^2$. We incorporate the regularizer as an additive term in the loss:

$$\mathcal{L}_{\mathrm{pc_reg}}(\lambda_t, \tilde{\boldsymbol{X}}; \alpha, \delta) = \alpha \max(\lambda_t / \|\tilde{\boldsymbol{X}}_t\|_\mathrm{F}^2 - \delta, 0), \tag{6}$$

where α is the strength of the regularizer and δ is the threshold proportion of variance explained at which it is imposed. In our experiments, we tune α as a hyperparameter with values in [0.1, 1, 10], and set $\delta = 0.2$ based on our analysis of the first principal components of models without the block structure. To speed up the training process, we only apply the regularizer to ReLU layers starting from the second stage, where block structure is often found.